TECHNICAL DOCUMENTATION · VERSION 1.0.0

# GlareTracker

Windshield glare tracking system
Code and program structure guide

**Purpose**
Explain how the webcam, MediaPipe face landmarks, geometry model, PyQt5 interface, 2D windshield view, and 3D scene work together.

**Project entry point**
`glare_tracker.py`

# Contents


1. System overview 3
2. Repository structure 3
3. Runtime data flow 4
4. Geometry and coordinate model 5
5. Python module structure 6
6. User interface and controls 8
7. Packaging and installation 9
8. Running, testing, and troubleshooting 10


# 1. System Overview

GlareTracker is a Windows desktop application. It estimates the driver's eye position from a webcam, creates rays from the eye toward one or more configurable light sources, and finds where those rays meet a virtual windshield.

**Webcam frame → MediaPipe iris landmarks → pixel-to-centimeter position → ray/plane intersection → 2D, 3D, and telemetry updates**

| Layer | Technology | Responsibility |
|---|---|---|
| Capture | OpenCV + QThread | Read frames without freezing the GUI. |
| Detection | MediaPipe Face Mesh | Locate iris landmarks 468 and 473. |
| Model | NumPy + helper functions | Convert coordinates and calculate intersections. |
| Interface | PyQt5 | Controls, camera preview, readout, and application lifecycle. |
| Visualization | QPainter + pyqtgraph OpenGL | Show glare positions and 3D rays. |

## 2. Repository Structure

| File or folder | Role |
|---|---|
| `glare_tracker.py` | Complete application: geometry, worker thread, widgets, state, signals, and entry point. |
| `requirements.txt` | Python dependencies required for development and packaging. |
| `glare_tracker.spec` | PyInstaller configuration; collects MediaPipe and pyqtgraph resources. |
| `build_exe.ps1` | Installs requirements, builds the executable, and optionally runs Inno Setup. |
| `installer.iss` | Inno Setup recipe for the per-user Windows installer. |
| `CreateDesktopShortcut.ps1` | Creates project-root and Desktop shortcuts to the packaged executable. |
| `generate_icon.py` | Regenerates the Windows icon from the SVG source. |
| `version_info.txt` | Windows executable version and product metadata. |
| `assets/` | Application icon files: ICO for Windows and SVG editable source. |

# 3. Runtime Data Flow

## Startup

1. `main()` creates `QApplication`, applies the Fusion style, constructs `MainWindow`, and starts the Qt event loop.
2. `MainWindow` creates all panels and the `VideoCaptureThread`, connects signals, pushes initial settings, and starts timers.
3. The scene is initially prepared with hidden/off-screen tracking markers.

## Frame processing

1. Pressing **Start Stream** starts the worker thread.
2. OpenCV opens camera index 0 using DirectShow, requests 640×480 at 30 FPS, and mirrors each frame.
3. MediaPipe processes an RGB copy. When a face is found, landmarks 468 and 473 are averaged into an iris midpoint.
4. The midpoint becomes an eye position in centimeters and is emitted through `eye_detected`.
5. The GUI callback applies calibration, calculates intersections, and updates every view.

## Signals and ownership

| Signal | Emitted by | Connected to | Meaning |
|---|---|---|---|
| `frame_ready(QImage)` | VideoCaptureThread | VideoWidget.update_frame | Display the latest camera frame. |
| `eye_detected(x,y,z)` | VideoCaptureThread | MainWindow._on_eye_detected | Deliver a detected eye position. |
| `eye_lost()` | VideoCaptureThread | MainWindow._on_eye_lost | Clear tracking visuals and show no-face status. |

**Threading rule:** camera capture and MediaPipe inference run in `VideoCaptureThread`. Qt signals transfer results to the GUI thread, so widgets are updated in the thread that owns them.

# 4. Geometry and Coordinate Model

The application uses centimeters and a right-handed conceptual scene:

- **X:** lateral position, left/right.
- **Y:** depth. The windshield is at `Y = 0`; the driver is normally behind it at negative Y; light sources are in front at positive Y.
- **Z:** vertical height.

## Pixel mapping

`pixel_to_cm()` maps the iris midpoint from the camera image into the configured windshield dimensions. Horizontal image center maps to X=0. Image Y is inverted so higher screen positions produce higher Z. Eye

depth is the configured negative `eye_dist_cm`.

$$x = (\text{pixel_x} - \text{frame_width} / 2) / \text{frame_width} \times \text{screen_width}$$
$$y = -\text{eye_distance}$$
$$z = (\text{frame_height} / 2 - \text{pixel_y}) / \text{frame_height} \times \text{screen_height} + \text{screen_height} / 2$$

## Ray intersection

`compute_intersection(eye_cm, light_cm)` parameterizes the line from eye $P_1$ to light $P_2$ as $P(t)=P_1+t(P_2-P_1)$. For the windshield plane Y=0:

$$t = -y_1 / (y_2 - y_1)$$
$$\text{intersection} = (x_1 + t(x_2-x_1),\ 0,\ z_1 + t(z_2-z_1))$$

The function returns `None` when the ray is parallel to the plane or when the crossing lies outside the segment (`t` is not between 0 and 1). A returned point is in range only when $-\text{width}/2 \le X \le \text{width}/2$ and $0 \le Z \le \text{height}$.

# 5. Python Module Structure

| Component | Purpose | Important methods |
|---|---|---|
| `TrackingConfig` | Immutable configuration passed to capture logic. | Dataclass defaults for dimensions, eye distance, and camera position. |
| `VideoCaptureThread` | Background camera and MediaPipe worker. | `run`, `stop`, `update_config`. |
| `VideoWidget` | Aspect-ratio-preserving camera preview. | `update_frame`, `clear_feed`. |
| `Windshield2DWidget` | Custom front-view drawing. | `update_glares`, `paintEvent`. |
| `Scene3DWidget` | OpenGL scene and visual settings overlay. | `update_eye`, `update_lights`, `update_rays`, `update_intersections`. |
| `ReadoutPanel` | Telemetry labels and intersection messages. | `set_value`, `set_intersections`. |
| `ControlsPanel` | Spin boxes, modes, and action buttons. | `get_light_positions`, dimension and camera getters. |
| `MainWindow` | Coordinator and application state owner. | `_process_tracking`, stream, calibration, simulation, and shutdown handlers. |

### Main tracking algorithm

```
_on_eye_detected(x, y, z)
  observed = (x, y, z)
  capture baseline on first observation, if needed
  subtract calibration offset
  _process_tracking(adjusted_eye)

_process_tracking(eye)
  choose simulated or configured lights
  update 3D eye, lights, and rays
  for each light:
    intersection = compute_intersection(eye, light)
    classify as mathematical miss, out of bounds, or valid
    convert valid/display points to normalized 2D coordinates
  update 3D intersections, telemetry, and 2D glare spots
```

### Calibration

The first detected eye is automatically used as a baseline. The X and Z offsets move that observed point to the windshield center; Y is intentionally left unchanged. The **Calibrate Center** button captures the latest observed eye again.

### Missing-face behavior

`_on_eye_lost()` clears the latest eye, moves 3D tracking objects off-screen, clears the 2D glare, empties intersection readouts, and reports `NO FACE`.

# 6. User Interface and Controls

## Five-panel dashboard

1. **Camera:** mirrored feed with iris overlays.
2. **Windshield front view:** proportional windshield rectangle, labels, glowing valid glare spots, and red out-of-range indicators.
3. **3D scene:** windshield plane, axes, webcam, eye, light sources, ray lines, and intersections.
4. **Telemetry:** eye coordinates, light mode, FPS, status, and per-light intersection results.
5. **Controls:** dimensions, camera location, driver distance, vehicle category, source position, movement simulation, stream, calibration, and reset.

## Light source modes

| Vehicle category | Sources |
|---|---|
| Motorcycle | One source at the configured position. |
| Four-Wheeler/Car | Two sources offset ±8 cm on X. |
| Extreme Vehicle | Four sources in a ±10 cm X and ±4 cm Z arrangement. |

## Movement simulation

The 50 ms movement timer translates the complete light arrangement while preserving the relative arrangement of sources. The three modes vary lateral crossing, approach direction, and distance. Simulation uses the same intersection pipeline as live tracking, so it is useful for checking visualization behavior without changing the geometry algorithm.

## Performance details

3D objects are created once and updated in place. Repeated positions are skipped using cached tuples. Unused source slots are moved to coordinates near 9999 so the scene supports up to four sources without repeatedly adding/removing OpenGL items.

# 7. Packaging and Installation

**requirements.txt → PyInstaller spec → dist/GlareTracker.exe → Inno Setup installer**

1. Run `./build_exe.ps1` in PowerShell.
2. The script installs dependencies from `requirements.txt`.
3. PyInstaller reads `glare_tracker.spec`, collects package data and hidden imports for MediaPipe and pyqtgraph, embeds the icon, and creates a windowed executable.
4. The executable is copied to the project root as `GlareTracker.exe`.
5. If `iscc.exe` is on PATH, Inno Setup creates `installer-output/GlareTracker-Setup.exe`.

The installer uses a per-user location under `%LOCALAPPDATA%\Programs\GlareTracker`, creates a Start Menu shortcut, and offers an optional Desktop shortcut. The standalone shortcut script points directly to the root

executable.

# 8. Running, Testing, and Troubleshooting

## Development run

```
python -m pip install -r requirements.txt
python glare_tracker.py
```

## Basic functional checklist

- Application starts with no import errors.
- Start Stream opens the webcam and shows a mirrored frame.
- A detected face changes status to `TRACKING` and updates eye coordinates.
- Moving light coordinates changes rays and intersection results.
- Changing the vehicle category changes the number of sources.
- Calibrate Center moves the current eye reference to the windshield center.
- Stop Stream releases the camera and clears visual tracking state.
- Closing the window stops timers and waits for the capture thread.

## Common issues

| Symptom | Likely cause and action |
|---|---|
| Camera feed stays stopped | Camera unavailable, permissions, or another application owns it. Check Windows camera privacy settings and camera index 0. |
| No face detected | Improve lighting, face the camera, and keep the eyes visible. MediaPipe needs a usable face image. |
| 3D view is blank | Check Windows graphics/OpenGL support and the PyOpenGL installation. |
| All intersections are out of range | Review windshield dimensions, eye distance, and light Y/Z positions. The ray must meet the plane inside its X/Z bounds. |
| Packaged app cannot find icon | Confirm `assets/glaretracker-icon.ico` is included by the spec and rebuild with `--clean`. |

**Model limitation:** pixel-to-centimeter conversion is an approximation based on the configured image-to-windshield dimensions and fixed eye distance. It is not a calibrated camera-pose or depth-estimation system.

**Good extension points:** move geometry helpers into a separate module, add unit tests for intersection edge cases, add camera calibration, and store user settings with `QSettings`.

# Quick Reference

| Question | Answer |
|---|---|
| Where does computation start? | `MainWindow._process_tracking()` |
| Where is the ray math? | `compute_intersection()` |
| Where does webcam work happen? | `VideoCaptureThread.run()` |
| Where are controls defined? | `ControlsPanel` |
| Where is packaging configured? | `glare_tracker.spec`, `build_exe.ps1`, and `installer.iss` |